02 / ALGEBRA / EX25

Guía de pseudocódigo

De un enunciado matemático al algoritmo y de ahí al programa.

DOCUMENTO 04 / PSEUDOCÓDIGO

Qué aprenderás

- Transformar un problema matemático en una secuencia comprensible.
- Identificar entradas, cálculos, salidas y controles.
- Escribir pseudocódigo antes de comenzar con NumPy.

Requisitos previos: Teoría, referencia y enunciado del EX25.

Orden de lectura: 1. Teoría → 2. Referencia → 3. Ejercicio → 4. Pseudocódigo → 5. Programación.

1. ¿Qué es el pseudocódigo?

El pseudocódigo es una **descripción ordenada del algoritmo**, escrita en lenguaje natural estructurado. No tiene que ser ejecutable ni seguir la sintaxis exacta de Python. Su misión es separar el razonamiento de la escritura de instrucciones.

> **Regla:** primero identifica qué debe calcularse; luego en qué orden; por último elige las funciones de Python. No empieces buscando comandos antes de entender las operaciones.

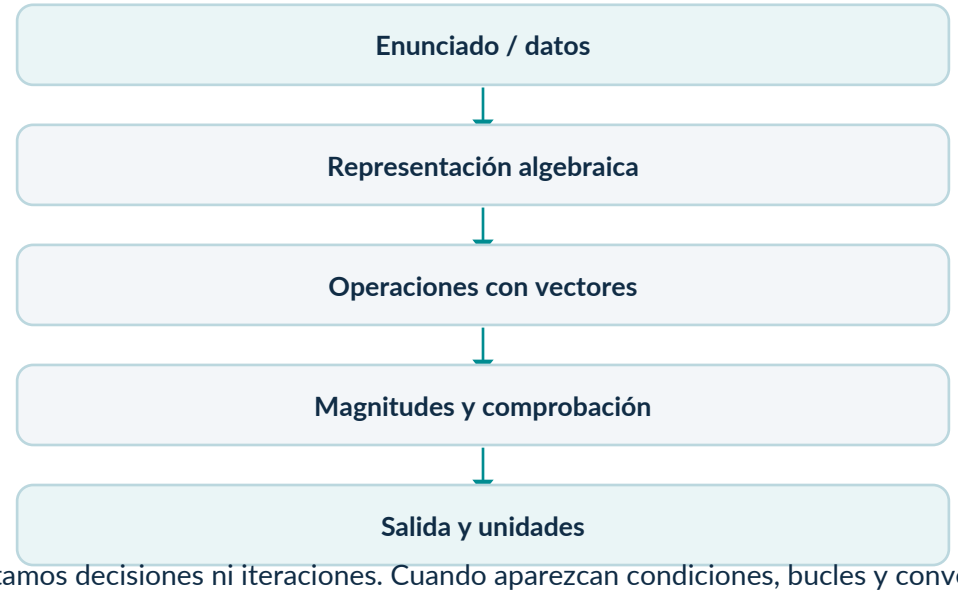
2. Cinco preguntas previas a cualquier algoritmo

- **¿Qué conozco?** Datos de entrada, unidades, dimensiones y sistema de referencia.
- **¿Qué quiero obtener?** Resultados concretos, vectoriales y escalares.
- **¿Qué matemáticas lo relacionan?** Fórmulas, propiedades y restricciones.
- **¿Qué pasos necesito?** Operaciones en el orden correcto.
- **¿Cómo compruebo?** Ejemplo manual, unidades, señales y tolerancias.

3. Conexión matemática → algoritmo → Python

Matemáticas	Pseudocódigo	Herramienta Python
Vector (a,b)	CREAR vector con 2 componentes	np.array(...)
Suma u+v	SUMAR componentes	u + v
Norma 2	CALCULAR longitud	np.linalg.norm(...)
Magnitud con unidad	MOSTRAR resultado	print(...)

4. Diagrama de flujo conceptual



Aún no necesitamos decisiones ni iteraciones. Cuando aparezcan condiciones, bucles y convergencia, añadiremos símbolos de decisión, ramas y retroalimentación.

5. Pseudocódigo resuelto: parte A

Partimos de los datos del ejemplo algebraico ya resuelto. El algoritmo representa en palabras lo que hemos calculado a mano.

```
INICIO
  DEFINIR u = (3, -1) y v = (-2, 4)
  CREAR dos vectores numericos
  CALCULAR suma = u + v
  CALCULAR combinacion = 2 * u - v
  CALCULAR norma_suma
  CALCULAR norma_combinacion
  MOSTRAR resultados
  COMPARAR con los calculos manuales
FIN
```

6. Tu pseudocódigo: parte B, fuerzas

Para el problema físico no te damos un programa de Python terminado. Completa un algoritmo utilizando esta plantilla y decide tú dónde calcular cada magnitud.

```
INICIO
  DEFINIR los datos de las fuerzas F1 y F2
  CONSTRUIR los vectores de componentes
  CALCULAR ... (magnitud primera fuerza)
  CALCULAR ... (magnitud segunda fuerza)
  CALCULAR ... (vector resultante)
  CALCULAR ... (magnitud resultante)
  MOSTRAR ... (con unidades)
  COMPROBAR ... (contra el calculo manual)
FIN
```

7. ¿Cómo validar un buen pseudocódigo?

- Cada paso tiene una única finalidad comprensible.
- Los resultados de un paso ya están disponibles cuando los usa el siguiente.
- Las variables vectoriales y las magnitudes escalares se distinguen.
- Las unidades de los datos y salidas se mantienen coherentes.
- Hay una comprobación independiente del código.

8. Errores típicos

Demasiado vago: “calcular fuerzas”. No especifica qué operación hace falta.

Demasiado pegado a Python: copiar una llamada de NumPy sin saber qué representa. El pseudocódigo debe poder explicarse también a mano.

Sin validación: imprimir un número sin comprobar su plausibilidad.

9. Plantilla general para próximos ejercicios

```
INICIO
  IDENTIFICAR datos, unidades y dimensiones
  DEFINIR variables y representaciones
  APLICAR las relaciones matematicas
  CALCULAR resultados intermedios
  VALIDAR unidades, signos y resultados
  MOSTRAR / GUARDAR resultados
FIN
```

En ejercicios posteriores enriqueceremos esta plantilla con **SI / ENTONCES**, **PARA**, **MIENTRAS**, validaciones de entrada y módulos reutilizables, siempre explicando primero su necesidad.